

✓ Q1 — What is an operating system? Four major tasks

中文回答：

操作系统（Operating System, OS）是管理计算机硬件并为应用程序提供服务的软件。
它的四大核心任务是：

1. 进程管理（Process Management）

- 创建、终止进程
- 进程调度
- 进程同步、通信、死锁处理

2. 内存管理（Memory Management）

- 分配/回收内存
- 地址转换（MMU、页表）
- 虚拟内存、分页、页面置换

3. 文件系统管理（File System Management）

- 管理文件、目录、权限
- 磁盘布局、缓存、索引节点
- 持久化存储与访问接口

4. 设备/I/O 管理（I/O Management）

- 管理设备驱动、缓冲区、DMA
- 中断处理
- I/O 调度与性能优化

English Answer:

An operating system is system software that manages hardware resources and provides services to applications.
Its four major tasks are:

1. **Process management:** creation/termination, scheduling, synchronization, IPC, deadlock handling.
2. **Memory management:** allocation, deallocation, address translation, virtual memory.
3. **File-system management:** file and directory handling, permissions, persistent storage.
4. **I/O and device management:** device drivers, interrupt handling, buffering, DMA, scheduling.

✓ Q2 — How to allow choosing OS at boot? What must bootstrap do?

中文回答：

可通过 **多重引导（Multi-boot）** 机制，例如 GRUB/UEFI Boot Manager。
Bootstrap（引导程序）需要：

1. 初始化硬件
2. 加载主引导加载器 (MBR/UEFI)
3. 显示系统列表供用户选择
4. 加载所选 OS 的内核到内存
5. 将控制权交给 OS 内核

English Answer:

A system can support multi-boot using a bootloader (e.g., GRUB).
The bootstrap program must:

1. Initialize hardware
2. Load the primary bootloader
3. Display a menu of installed OS choices
4. Load the selected OS kernel into memory
5. Transfer control to the chosen kernel

✓ Q3 — Microkernel: advantage, communication, disadvantage

中文回答：

主要优点：

- 内核更小、更可靠
- 模块化强，易扩展、易维护
- 服务在用户态崩溃时不会击穿内核

用户程序与系统服务如何通信？

- 通过消息传递 (Message Passing IPC)

缺点：

- 频繁的 IPC 开销大
- 系统调用延迟更高
- 性能通常不如单体内核 (Monolithic Kernel)

English Answer:

Advantage:

Higher reliability, modularity, maintainability; most services run in user mode.

Interaction:

User programs and services communicate via **message passing**.

Disadvantages:

High IPC overhead, slower system calls, lower performance compared to monolithic kernels.

✓ Q4 — Why layered OS? Pros & cons

中文回答：

为什么采用分层设计？

为了将复杂系统模块化，提高可维护性和可理解性。

优点：

- 结构清晰
- 易调试、易扩展、易维护
- 每层只依赖下层 API，安全性高

缺点：

- 不同层之间的调用链会降低性能
- 层次划分难（层间接口设计困难）

English Answer:

Advantages:

Clear structure, modularity, easier debugging and maintenance.

Disadvantages:

Potential performance overhead, difficulty in defining proper layers.

✓ Q5 — What is a process? What is a thread? Compare them

中文回答：

进程 (Process)：

资源分配单位，拥有独立的地址空间、文件描述符、代码/数据段。

线程 (Thread)：

CPU 调度单位，一个进程可以有多个线程共享地址空间和资源。

比较：

- 线程创建开销小，切换快
- 线程共享资源但带来同步风险
- 进程隔离性强但创建开销大

English Answer:

A **process** is a resource container with its own address space.

A **thread** is a lightweight execution unit inside a process.

Comparison:

- Threads are cheaper to create and switch
 - Threads share resources → need synchronization
 - Processes isolate resources → more overhead
-

✓ Q6 — Resources required when a thread is created

中文回答：

创建线程需要：

- 线程栈 (Stack)
- 程序计数器 (PC)、寄存器上下文
- 线程控制块 (TCB)
- 调度信息 (优先级、运行状态)

English Answer:

Thread creation requires:

- A thread stack
 - Program counter and register context
 - A Thread Control Block (TCB)
 - Scheduling information (priority, state, etc.)
-

✓ Q7 — Five states of a process & their relationships

中文回答：

五大进程状态：

1. **new** (新建)
2. **ready** (就绪)
3. **running** (运行)
4. **waiting** (阻塞)
5. **terminated** (终止)

状态关系：

```
new → ready → running → terminated
                        ↓
                        waiting → ready
```

English Answer:

The five states are new, ready, running, waiting, and terminated.

Transitions:

- new → ready → running
 - running → waiting (on I/O) or ready (preempted)
 - running → terminated
-

✓ Q8 — Actions taken during a context switch

中文回答：

上下文切换时内核需要：

1. 保存当前进程的寄存器内容、PC 到 PCB
2. 更新调度器数据结构
3. 加载下一个进程的 PCB 信息
4. 恢复寄存器和 PC
5. 切换页表 (MMU)
6. 将 CPU 执行权交给新进程

English Answer:

During a context switch, the kernel:

1. Saves the current process context (registers, PC)
2. Updates scheduler structures
3. Loads the next process's PCB
4. Restores registers and PC
5. Switches address space/page tables
6. Resumes execution of the new process

✓ Q9 — Why user mode & kernel mode? Pros & cons

中文回答：

原因：保护系统安全，防止用户程序直接操作硬件。

优点：

- 安全性高，防止崩溃影响整个系统
- 隔离关键资源

缺点：

- 系统调用需要模式切换，成本高
- 用户态无法直接完成高级硬件操作，需要调用内核 API

English Answer:

Reason: To protect hardware and kernel resources.

Pros:

High security, isolation, better system stability.

Cons:

Mode switching overhead, less direct hardware control for user programs.

✓ Q10 — OS activities for memory mgmt & secondary storage mgmt

中文回答：

内存管理的三个主要活动：

1. 内存分配与回收（给进程分配空间）
2. 地址转换（虚拟地址→物理地址）
3. 页面置换（Page Replacement）

外存管理的三个主要活动：

1. 空闲空间管理（Free space management）
2. 磁盘调度（Disk scheduling）
3. 文件组织与存储分布管理

English Answer:

Memory management (three activities):

1. Allocation and deallocation
2. Address translation
3. Page replacement and virtual memory handling

Secondary storage management (three activities):

1. Free-space management
2. Disk scheduling
3. File organization and layout management

✓ Q11 — Can segmentation and paging work together? Why?

中文回答：

可以。

例如 **x86 架构** 同时使用“分段 + 分页”机制：

- **段 (Segment)** 提供逻辑结构（代码段、数据段、栈段）
- **页 (Page)** 提供物理内存管理（固定大小、避免外部碎片）

二者合作优势：

- 分段体现程序逻辑结构，可实现保护与共享
- 分页避免外部碎片，简化内存分配
- 二者结合更灵活、安全、兼具性能与结构化内存模型

English Answer:

Yes.

For example, x86 uses both segmentation and paging.

- **Segmentation** provides logical structure and protection
- **Paging** provides physical memory allocation and avoids external fragmentation

Together they offer both logical organization and efficient memory management.

✓ Q12 — What is Thrashing? How to resolve it?

中文回答：

Thrashing (抖动) 是指系统频繁缺页，使 CPU 大部分时间都在进行页面换入换出，进程几乎无法正常执行。

解决方法：

1. **工作集模型 (Working-Set Model)**
保证进程活跃页集都在内存中。
2. **PFF (Page Fault Frequency) 控制**
缺页率过高 → 分配更多页；缺页率过低 → 回收页。
3. **减少多道程序度 (限制并发进程数)**
4. **增加物理内存**

English Answer:

Thrashing is excessive paging activity that prevents useful execution.

Solutions include:

- Working-set model
 - Page-fault frequency control
 - Reducing degree of multiprogramming
 - Adding more physical memory
-

✓ Q13 — What is TLB? How does it work?

中文回答：

TLB (Translation Lookaside Buffer) 是**页表缓存**，用于加速虚拟地址到物理地址的转换。

工作流程：

1. CPU 提供虚拟地址
2. 查询 TLB

- **命中 (hit)** : 直接得到物理页号
- **未命中 (miss)** : 查页表 → 更新 TLB

3. 得到物理地址后访问内存

TLB 是内存管理的关键加速部件。

English Answer:

The TLB is a cache that stores recent virtual→physical page translations.

How it works:

1. CPU generates a virtual address
2. TLB lookup
 - Hit: translation found
 - Miss: page table consulted, TLB updated
3. Access memory using physical address

✓ Q14 — Purposes of TLB + how it works with L1/L2/L3 cache

中文回答：

TLB 的主要目的：

1. 加速地址转换
2. 减少页表访问
3. 降低内存访问延迟
4. 提升 CPU 吞吐量

在含 L1/L2/L3 cache 的系统中：

流程如下：

```
CPU → TLB → physical address → L1 cache → L2 → L3 → memory
```

解释：

- TLB 提供物理地址
- Cache 层级用于存储数据
- TLB 属于“地址转换路径”，Cache 属于“数据访问路径”
- 二者互不替代，但组合提高整体性能

English Answer:

Purposes of the TLB:

- Speed up address translation

- Reduce page-table lookups
- Improve overall memory-access performance

With multi-level caches:

The flow is:

```
CPU → TLB → PA → L1 → L2 → L3 → memory
```

The TLB maps VA→PA; caches store data at those physical addresses.

✓ Q15 — What is an interrupt? What are its usages?

中文回答：

中断 (Interrupt) 是一种异步事件机制，用于让 CPU 响应设备或软件的请求。

用途：

1. **I/O 完成** (例如 DMA 完成)
2. **系统调用** (软中断)
3. **异常/错误处理** (如除零错误)
4. **时钟中断** (用于调度)
5. **设备事件** (键盘、网络等)

English Answer:

An interrupt is a signal that transfers control to the OS.

Uses include:

- I/O completion
- System calls
- Exceptions
- Timer/scheduling
- Hardware device notifications

✓ Q16 — How to use interrupt & polling for process manipulation?

中文回答：

轮询 (Polling) 机制：

- CPU 主动循环检查设备状态
- 简单但浪费 CPU 时间
- 适用于高速设备或短时间检查

中断 (Interrupt) 机制：

- 设备主动通知 CPU
- CPU 不需反复检查
- 适用于低速设备和高效率系统

在进程控制中：

- 轮询：进程主动检查 I/O 完成
- 中断：I/O 完成后触发中断 → OS 进行调度、唤醒阻塞进程

English Answer:

Polling: CPU repeatedly checks device status; simple but inefficient.

Interrupts: device notifies CPU, more efficient.

For process control:

- Polling-based processes manually check for completion
- Interrupt-driven I/O wakes blocked processes and triggers scheduler

✓ Q17 — What is Copy-on-Write? How does it work? Advantages?

中文回答：

写时复制 (Copy-on-Write, COW) 是在 `fork()` 时父子进程共享只读页，只有在写入时才进行复制。

工作原理：

1. `fork` 时父子共享相同的物理页
2. 页标记为只读
3. 任一进程尝试写入 → 触发页错误
4. OS 为写操作复制一份新的物理页
5. 两者之后独立使用不同的页

优点：

- 节省内存
- 加速 `fork` (不复制整块地址空间)
- 提高性能

English Answer:

Copy-on-Write allows forked processes to share memory until a write occurs.

Advantages:

- Saves memory
- Makes `fork` fast
- Reduces overhead of duplicating memory unnecessarily

✓ Q18 — Why DMA? How does DMA transfer data?

中文回答：

为什么需要 DMA？

让 I/O 设备在无需 CPU 参与的情况下直接读写内存，提高吞吐量。

DMA 传输步骤：

1. CPU 设置 DMA 控制器（源地址、目的地址、大小）
2. DMA 开始直接内存传输
3. CPU 可执行其他任务
4. DMA 完成后发中断 → 通知 CPU

English Answer:

DMA enables devices to directly read/write memory without CPU intervention.

Flow:

1. CPU programs the DMA controller
2. DMA performs transfer directly
3. CPU works concurrently
4. DMA raises an interrupt when done

✓ Q19 — Memory-Mapped I/O and file access

中文回答：

Memory-Mapped I/O 使用 `mmap()` 将文件部分映射到虚拟内存。

执行文件读取/写入步骤：

1. `mmap` 创建“文件页 ↔ 内存页”的映射
2. 访问内存 = 访问文件
3. 缺页 → OS 从磁盘读入对应页
4. 写操作（`MAP_SHARED`）会回写磁盘

优点：更高效、更简单的文件访问方式。

English Answer:

Memory-mapped I/O maps files into virtual memory.

Accessing memory automatically accesses file pages; page faults load them from disk, and writes can propagate back to the file.

✓ Q20 — Design a file system efficient for both small & large files

中文回答：

一个同时适合小文件和大文件的文件系统需要混合设计：

对小文件：

- 使用 **直接块 (direct blocks)**
- 小文件可直接放在 inode 中（如 ext4 inline data）

对大文件：

- 使用 **extent (连续块)** 技术
- 或使用 B+ 树 / extent tree 来组织大文件

结合策略使 FS 兼顾两者：

- 小文件 → 少量 direct blocks + 低寻道开销
- 大文件 → extent + 分配大连续区，提高顺序读取性能

Ext4、XFS、APFS 都使用这种混合策略。

English Answer:

A hybrid file-system design works best:

- **Small files:** use direct blocks or inline data for fast access
- **Large files:** use extents or B-tree-based extent trees

✓ Q21 — Advantages of FAT-based linked allocation

中文回答：

FAT（File Allocation Table）是把链式分配的“链表指针”全部放进内存中的表结构，而不是散落在磁盘块里。

其优点：

1. 快速遍历文件块链

- 原始链式分配需要读取每个磁盘块才能看到“下一指针”。
- FAT 直接在内存表中查下一块→不需要磁盘访问。

2. 顺序访问极快

- 只需沿着 FAT 表的链即可，不必磁盘跳转。

3. 文件碎片问题较轻

- 即使块不连续，也能快速遍历链。

4. 实现简单、兼容性强

- FAT 文件系统非常轻量，适合嵌入式系统、U 盘、SD 卡等。

English Answer:

The advantages of using FAT for linked allocation include:

1. Fast traversal

Block pointers are in memory, avoiding disk accesses.

2. Efficient sequential access

Only need to follow the in-memory FAT chain.

3. Less impact from fragmentation

Non-contiguous blocks don't significantly hurt traversal.

4. Simple and widely supported

FAT is easy to implement and widely used in portable storage.

✓ Q22 — mmap role; MAP_SHARED vs MAP_PRIVATE; ref counter changes

中文回答：

mmap 的作用：

- 把文件的内容映射到进程虚拟内存
- 内存访问 = 文件访问
- OS 管理缺页并自动加载对应文件页
- 提高 I/O 效率（无需 read/write 系统调用）

MAP_SHARED vs MAP_PRIVATE：

类型	行为	是否回写磁盘
MAP_SHARED	修改内容会同步到文件	✓ 是
MAP_PRIVATE	写入触发 COW，只影响私有副本	✗ 否

引用计数何时增减？

- mmap 时增加引用计数（文件被映射一次，引用 +1）
- munmap 时减少引用计数
- fork 时也会增加引用计数（父子共享相同映射）

当引用计数降为 0 时，OS 可以关闭映射、释放页缓存。

English Answer:

Role of mmap:

Maps a file into virtual memory, allowing file access via memory loads/stores.

MAP_SHARED:

Writes propagate to the underlying file (shared mapping).

MAP_PRIVATE:

Copy-on-write; modifications are private and not written back.

Reference counter:

Increment on mmap or fork; decrement on munmap. When it reaches zero, the mapping can be fully released.

✓ Q23 — When LFU beats LRU, and vice versa

中文回答：

LFU 优于 LRU 的情况：

- 存在“长期热点数据”
- 虽然最近使用较少，但历史访问频率高
例如：
- 配置文件、常驻库、长期稳定的热门 key

LRU 在这种情况下会把“最近未访问但长期热门”的页淘汰掉，而 LFU 不会。

LRU 优于 LFU 的情况：

- 工作集有“频繁变化的热点”
- 程序具有强烈的时间局部性 (temporal locality)
例如：
- 分页程序访问序列快速变化 (旧热点已不相关)

此时 LFU 会因为历史频率导致“旧热点”仍占据缓存位置，反而降低命中率。

English Answer:

LFU wins when:

- Long-term frequently used items dominate
- Historical frequency is more important than recent use

LRU wins when:

- Temporal locality is strong
 - The hot set changes rapidly
 - Recent usage is more important than old behavior
-

✓ Q24 — Life Cycle of an I/O request

中文回答：

I/O 请求的完整生命周期如下：

1. 应用调用系统调用 (read/write/mmap 等)
2. OS 将请求提交给 I/O 子系统
3. I/O 调度器重新排序请求 (如 elevator / SSTF / C-SCAN)
4. 设备驱动接手请求
5. 若支持 DMA → DMA 启动数据传输
6. 设备完成传输, 产生中断
7. OS 处理器中断, 更新 I/O 状态
8. 唤醒等待该 I/O 的进程
9. 应用得到返回结果并继续执行

English Answer:

Life cycle:

1. Application issues syscall
2. OS submits request to I/O subsystem
3. I/O scheduler reorders queue
4. Device driver processes request
5. DMA (if available) performs data movement
6. Device raises interrupt upon completion
7. OS handles interrupt and updates status
8. Waiting process is awakened
9. Application resumes execution

✓ Q25 — Larger page size (128 KB)? Pros & cons

中文回答：

优点 (Pros)：

1. 更少的页表项 (page table entries)
2. TLB 命中率更高 (TLB reach 变大)
3. 减少页表查找开销
4. 适合大数据处理 (如数据库、ML 工作负载)

缺点 (Cons)：

1. 内部碎片更大 (浪费内存)
2. 对于“多数小进程”不友好
3. 加载大页需要一次性读更多数据 (缺页代价变贵)
4. 难以进行细粒度的内存保护和共享

English Answer:

Pros:

- Fewer page table entries
- Higher TLB reach
- Less overhead in page-table traversal
- Good for large memory workloads

Cons:

- Larger internal fragmentation
- Worse for small processes
- Faults become more expensive
- Coarse-grained protection

Q25(b) — Impact on cache, TLB, memory management

中文回答：

- **Cache**：大页可能降低 cache locality（一次加载太多无关数据）
- **TLB**：TLB 未命中少了（好处）；但每条 TLB entry 覆盖更大区域
- **内存管理**：
 - 更容易导致内存碎片
 - 需要新的分配策略（buddy 或 huge pages）

English Answer:

- **Cache**: potentially worse locality
- **TLB**: better TLB efficiency and reach
- **Memory management**: increased fragmentation, more complex allocation policies

✓ Q26 — How to speed up access to frequently used records in triple-indirect large files

中文回答：

大文件（>4GB）需要 **三重间接块**，访问最后部分日志会产生多次磁盘寻道。
优化策略如下：

1. 使用页缓存（Page Cache）缓存最近写入的尾部块

这是 OS 最常用的优化方式。

2. 使用 Extent-Based 文件系统（如 ext4、XFS）

- 使用连续区块存储（extent）取代多级间接
- 直接定位大段连续区域
- 大幅减少磁盘寻道

3. 日志结构文件系统（LFS）

- 将尾部写入顺序化
- 访问最新记录只需读最后一个 segment

4. 文件系统预读（Read-Ahead）

OS 会预测顺序访问，提前将后续块读入缓存。

5. 使用索引结构 (B+ Tree)

APFS、XFS 等文件系统使用 B-tree 加速定位大文件末尾。

English Answer:

To accelerate accesses near the end of a large triple-indirect file:

- Cache recent blocks in the page cache
- Use an extent-based file system
- Adopt log-structured FS
- Enable read-ahead
- Use B-tree-based metadata indexing

✓ Q27 — Page table implementations: multi-level, hashed, inverted (pros & cons)

(1) Multi-level page tables

中文：

优点：

- 只需为实际使用的虚拟空间分配页表（节省内存）
- 很适合稀疏地址空间

缺点：

- 地址转换需要多个内存访问（除非 TLB 命中）
- 深度大的多级结构带来额外延迟

English:

Pros:

- Saves memory due to sparse allocation
- Easy to grow/shrink

Cons:

- Multiple memory accesses
- More levels = more latency

(2) Hashed page tables

中文：

优点：

- 适用于超大虚拟地址空间（如 64-bit）
- 查找理论上接近 $O(1)$

缺点：

- 哈希冲突导致链表遍历
- 实时性能可能不稳定
- 不适合有强局部性的工作负载（LRU 友好性差）

English:

Pros: good for large sparse address spaces, fast expected lookup

Cons: collisions cause overhead; less predictable performance

(3) Inverted page table

中文：

每个物理页只有一个表项，节省大量内存。

优点：

- 页表大小固定（与物理内存成比例）
- 内存消耗最小

缺点：

- 查找复杂 → 需要哈希
- 反向结构难以支持共享内存
- 虚拟地址到物理地址不再直接索引，需要额外结构加速

English:

Pros:

- Small, fixed-size table
- Memory-efficient

Cons:

- More complex lookup
 - Harder to support shared pages
 - Requires hash and TLB for speed
-